

Déployer un serveur Pi-Hole

Définition du projet

Pi-hole® est un filtre DNS qui protège tout appareil, de domaines connus sur des listes pré-établies, de tout contenu non désiré et sans installer aucun logiciel tiers.

Il s'agit là d'un premier projet en autodidacte afin de concrétiser l'apprentissage des lignes de commandes et apprendre à dépasser les a priori du terminal, la compréhension du système Linux ainsi qu'une mise en pratique des connaissances acquises lors d'un module TCP/IP & Virtualisation.

Objectifs

1. **Installer une VM sous Debian 12** : Sous Debian et sans interface graphique le projet réside en deux objectifs. Initier l'utilisateur aux lignes de commande uniquement, avec une première approche pour des utilisations futures (maintenance, sécurisation) et développer une maîtrise de la logique qui sera engagée.
2. **Déployer un Pi-Hole** : Le déploiement étant effectué sans aucune pré installation et strictement sous forme de lignes de commandes, permet d'accentuer l'apprentissage des OS sans interface graphique et concrètement combattre toute forme de défiance de la ligne de commande. La connexion SSH deviendra la référence pour la maintenance et l'optimisation du projet à l'avenir avec des tentatives de connexion externe.
3. **Appropriation du projet** : Sur du moyen terme et selon la maîtrise acquise, j'aimerais déployer ce projet au sein de mon propre réseau domestique et intégrer le pi-hole comme un DNS qui sécurise mon usage ainsi que celui de mes enfants dans le but de les protéger de tout contenu non approprié ou non sollicités.
UPDATE : Ce projet est désormais concrétisé mais il m'est impossible de me servir du routeur domestique de Bouygues comme d'un serveur d'entreprise. De ce fait je devrais utiliser mon Pi-Hole comme serveur DHCP mais pour éviter tout problème superflu le jour où il y aura une panne j'ai donc choisi de contourner l'inconvénient en configurant tous mes appareils sur le DNS du Pi-hole.
4. **Finalité** : Le projet de la VM sera reporté sur un Raspberry Pi 1 modèle B+ (v1.2), accessible pour quelques euros, en tenant compte des configurations réseaux nécessaires. Créer un serveur dédié sans devoir laisser allumé ni saturer le PC de la maison ni le PC portable et continuer la maintenance toujours en SSH.

Caractéristiques

Ma VM par le biais de l'Hyperviseur VMWare Workstation est un OS Debian 12. Elle comprend les caractéristiques suivantes :

- 2Go Ram
- 20 Go Disque
- 2 Cpu / 1 Coeur
- 1 carte réseau en NAT qu'il faudra configurer en BRIDGE pour répliquer la connexion de l'hôte.

Installation du serveur

1. Installation de Debian 12 à partir d'un Iso (EN)

Fichier : [debian-12.0.0-amd64-netinst.iso](#)

Virtualisation sous VMWare Workstation (gratuit depuis 2024)

Pour une totale évolution du projet ligne de commande on sélectionne uniquement les deux dernières options lors de l'installation (SSH Server & System user). Pour davantage de solutions trouvées sur le net et selon votre niveau d'anglais il est utile d'installer votre OS en Anglais (attention lors de la configuration du clavier, installez le en français).

Laissez-vous ensuite guider par l'installation.

Choose software to install:

- ☐ Debian desktop environment
- ☐ ... GNOME
- ☐ ... Xfce
- ☐ ... GNOME Flashback
- ☐ ... KDE Plasma
- ☐ ... Cinnamon
- ☐ ... MATE
- ☐ ... LXDE
- ☐ ... LXQt
- ☐ web server
- ☒ SSH server
- ☒ standard system utilities

2. Préparation du champ de travail

Ici on commence par les fondations. On commence par des règles qui seront utiles sur tout projet Linux et la plus importante selon moi est d'appliquer le principe du "moindre privilège" (**ne jamais travailler en root**).

- a. **Installation de sudo** : (Si absent lors de l'install)
Commande (en root `su -`) : `apt install sudo`
- b. **Élévation de l'utilisateur nicolas** :
Commande : `usermod -aG sudo nicolas`
Explication : `-a` (ajouter) et `-G` (groupe secondaire). Cela permet à `nicolas` d'utiliser les droits d'administration sans connaître le mot de passe root. Et ça c'est bien 👍
- c. **Vérification** :
Commande : `groups nicolas` (le mot `sudo` doit apparaître).

```
nicolas : nicolas cdrom floppy sudo audio dip video plugdev users netdev
```

3. Vérification de l'identité réseau

Avant toute chose, on identifie la machine sur le réseau.

Commande : `ip a`

Détail : Repérer l'interface (souvent `ens33`) et l'adresse IPv4 (ex: `192.168.x.x`) qui est en théorie configurée en *dynamic*.

Justification : Indispensable pour la future configuration de Pi-hole qui nécessite obligatoirement une IP statique. Autrement dit on oublie le DHCP même si c'est confortable et on va directement mettre les mains dans le fichier réseau un peu plus bas.

En dynamic : Durée du bail de 86330 secondes

```
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen
1000
    link/ether 00:0c:29:xx:xx:xx brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 192.168.1.126/24 brd 192.168.1.255 scope global dynamic ens33
        valid_lft 86330sec preferred_lft 86330sec
    inet6 2001:861:5680:3990:xxxx:xxxx:xxxx:xxxx/64 scope global dynamic mngtmpaddr
        valid_lft 86330sec preferred_lft 14330sec
    inet6 fe80::xxxx:xxxx:xxxx:xxxx/64 scope link
        valid_lft forever preferred_lft forever
```

Sur la capture (depuis mon accès SSH) on voit que l'adresse IP de la carte réseau (nommée ens33) a été accordée grâce au DHCP. Les indices sont la *durée du bail en secondes* et le mot *dynamic*.

[Petit aparté], dans certains cas, lors de mon apprentissage sur deux cartes réseaux avec moult essais j'ai même dû modifier les paramètres sous Windows (Windows+R et la commande `ncpa.cpl`).

Ne vous faites pas tout va bien se passer sur un raspberry on n'a pas besoin de tout ça, sur la VM en revanche j'ai fait les erreurs à votre place ! 😊

Ici, comme VMware est bien fait il faudra juste s'assurer que la carte réseau (généralement Vm0) sous Debian est bien en mode Bridge pour la virtualisation (sans oublier de répliquer la connexion) tandis que sur le raspberry c'est plus simple, on configure une IP statique sur le fichier carte réseau et on est dans la place !
[Fin]

4. Configuration de la carte réseau

De retour sur le projet/VMWare/Debian.

On va donc modifier le fichier réseau qui regroupe l'ensemble des cartes réseau (une seule ici) en tapant la commande suivante :

```
sudo nano /etc/network/interfaces
```

```
# This file describes the network interfaces available on your system
# and how to activate them. For more information, see interfaces(5).

source /etc/network/interfaces.d/*

# The loopback network interface
auto lo
iface lo inet loopback

# The primary network interface
allow-hotplug ens33
iface ens33 inet dhcp
```

Sur la capture, ens33 (ma carte réseau) fonctionne sur un modèle DHCP. Avant de modifier ce fichier, on va travailler sur l'hyperviseur.

Étape 1 sur VMWare :

- Dans VMware, faire un clic droit sur la VM Debian puis sélectionnez **Paramètres** (ou allez dans l'onglet VM > Paramètres ça marche aussi).
- **Sélectionner la carte réseau** : Dans l'onglet **Matériels**, sélectionnez votre **Carte réseau**.
- Cocher l'option **Pont : connecté directement au réseau physique**
- Cocher l'option **Répliquer l'état de la connexion réseau physique** pour que la VM puisse suivre les déconnexions et reconnexions au réseau.

Vous avez désormais une carte réseau virtuelle pontée sur votre propre réseau. *Sur le papier* votre VM est connectée à internet. Maintenant qu'on a donné le feu vert à VMware on va briefer la carte réseau sur sa nouvelle affectation avec une **IP statique**.

Étape 2 sur Debian :

Retour sur le fichier `etc/network/interfaces` précédé de la commande `sudo nano` qui permet d'ouvrir le fichier de configuration du réseau.

```
# This file describes the network interfaces available on your system
# and how to activate them. For more information, see interfaces(5).

source /etc/network/interfaces.d/*

# The loopback network interface
auto lo
iface lo inet loopback

# The primary network interface
# Traduction pour les noobs :
# adresse de la carte réseau (coucou internet, coucou les paquets on habite là et on attend la partie de
ping-pong)
# sous-réseau pour compléter l'adresse (bah oui faut bien faire comprendre à ton ordinateur ultra tatillon
c'est quoi son réseau, un peu comme le facteur perdu dans ton quartier car il n'a pas ton numéro de
maison)
# la passerelle qui relie ta carte réseau à internet (imagine le bateau dans The Truman Show, sans celui-ci
Truman serait resté isolé dans cette bulle. J'espère pour toi que t'as la ref, sinon éteint tout et va regarder
cette pépite!) Oui je peux littéralement marquer n'importe quoi ici et tu vas quand même lire.
# l'annuaire internet, plus gros que les pages jaunes mais il fait sans doute moins mal. On a utilisé le DNS
Cloudflare et Google.
auto ens33
iface ens33 inet static
    address 192.168.1.126
    netmask 255.255.255.0
    gateway 192.168.1.1
    dns-nameservers 1.1.1.1 8.8.8.8
```

Troubleshooting : Sur la capture ci-dessus j'avais commis une erreur sans même m'en rendre compte, j'ai indiqué le CIDR (/24 à la fin de l'IP `192.168.1.126/24`) sauf que cette information peut entrer en conflit avec le netmask renseigné juste en dessous et apparemment générer des soucis pour rendre la carte réseau active et ce malgré la commande `sudo ip link set ens33 up` qui comme le mot `up` l'indique, sert à mettre en marche (la carte réseau `ens33`).

Bon à savoir :

VMnet8 est obligatoirement configurée en NAT et avec un réseau différent de votre machine.

VMnet0 est configurée en Ponté sur la carte réseau avec une IP qui possède une adresse unique pour éviter tout conflit d'IP avec votre machine hôte.

Comme je travaille sur ce projet en intermittence j'ai déployé cette solution 3 mois après le début de cette doc, je me suis littéralement emmêlé les pinceaux et j'ai créé mon premier conflit d'IP ! Résultat je n'avais plus accès à rien. Comme je savais que ça venait de la VM j'ai pu isoler le problème rapidement et j'ai retrouvé l'usage de mon réseau.

Bref, maintenant qu'on a dit à VMware : on veut une connexion pontée sur le réseau de l'hôte et qu'on a configuré en statique la carte réseau de Debian, on obtient ceci en retapant la fameuse commande `ip a show ens33`.

Voici désormais la confirmation que vous avez une carte avec une IP statique : **Preferred left Forever** (forever remplace ici la notion de durée du bail accordé par le DHCP).

```
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
  link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
  inet 127.0.0.1/8 scope host lo
    valid_lft forever preferred_lft forever
  inet6 ::1/128 scope host noprefixroute
    valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen
1000
  link/ether b8:27:eb:xx:xx:xx brd ff:ff:ff:ff:ff:ff
  inet 192.168.1.126/24 brd 192.168.1.255 scope global ens33 # TIPS : On prend une IP hors de la pool
  valid_lft forever preferred_lft forever                                adress du serveur DHCP
  inet6 fe80::xxxx:xxxx:xxxx:xxxx/64 scope link
    valid_lft forever preferred_lft forever
```

Pour s'assurer que la machine résolve toujours les noms de domaine, vérifiez ou modifiez le fichier de configuration DNS :

```
sudo nano /etc/resolv.conf
```

Ajoutez-y l'adresse de votre passerelle ou un DNS externe si besoin :

```
nameserver 192.168.x.x
```

Pour prendre en compte la nouvelle configuration sans redémarrer la machine virtuelle, relancez le service réseau :

```
sudo systemctl restart networking
```

Si la carte ne remonte pas automatiquement, forcez l'activation :

```
sudo ip link set ens33 down && sudo ip link set ens33 up
```

Le fameux **state UP** qu'on voit sur la capture juste au-dessus.

Durcissement du service SSH (Via Terminal Windows)

On transforme la porte d'entrée standard (port 22) en une entrée sécurisée et discrète. *Je suis caché 🙈!*

1. Modification de la configuration

On édite le fichier maître : `sudo nano /etc/ssh/sshd_config`.

Changement de port : Choisir une valeur entre **1024 à 49151** : Les ports **Utilisateurs** (ou "Registered ports").

Pourquoi ? Évite les "scripts kiddies" et les robots qui scannent massivement le port 22.

Interdiction du login Root : `PermitRootLogin no`

Pourquoi ? Force un attaquant à trouver d'abord ton nom d'utilisateur (**nicolas**) avant de tenter de casser le mot de passe.

Limitation des tentatives : `MaxAuthTries 3`

Pourquoi ? Bloque les attaques par force brute (essais successifs de mots de passe).

Limitation des sessions : `MaxSessions 2`

Pourquoi ? Préviend la saturation des ressources du service SSH.

Ce qui donne dans le fichier

```
Include /etc/ssh/sshd_config.d/*.conf

Port 13250 # Petit clin d'oeil de chez moi
#AddressFamily any

#J'ai enlevé le superflu et j'ai laissé de quoi vous repérer. Uniquement les lignes blanches doivent être
modifiées

#LoginGraceTime 2m
PermitRootLogin no
#StrictModes yes
MaxAuthTries 3
MaxSessions 2
```

2. Application et Validation

On redémarre et on contrôle

Restart : `sudo systemctl restart ssh`

Contrôle : `ss -tulpn | grep 13250` (c'est un exemple bien sûr)

Analyse : On doit voir **LISTEN** sur le port choisi **13250**.

```
tcp  LISTEN 0      128        0.0.0.0:13250  0.0.0.0:*
tcp  LISTEN 0      128        [::]:13250    [::]:*
```

Mise en place du Pare-feu (UFW)

Le pare-feu **UFW** (*Uncomplicated Firewall*) agit comme une barrière de sécurité entre l'interface réseau et le système d'exploitation.

1. Installation de l'outil

Par défaut, Debian n'a pas de pare-feu actif. On installe donc l'outil :

```
sudo apt install ufw -y
```

2. Configuration des règles (Allow)

Avant d'activer le mur, on définit les exceptions. Si on oublie cette étape, on se bloque soi-même l'accès SSH... "non mais allow quoi!".

```
sudo ufw allow 13250/tcp
```

 (port/protocole)

Comme le service SSH s'appuie exclusivement sur la fiabilité du protocole TCP pour garantir l'intégrité et l'ordre des commandes envoyées. Cela permet également d'éviter d'ouvrir un port inutilement qui pourrait entraîner une attaque type déni de service.

3. Activation du service

Mise en place du pare-feu.

```
sudo ufw enable
```

Un message d'alerte s'affiche. Il faut confirmer par **y**. À ce stade, tous les ports sont fermés, sauf le **13250**.

4. Vérification et Audit

Un bon administrateur vérifie toujours que sa commande a eu l'effet escompté.

```
sudo ufw status verbose
```

Analyse du résultat :

Status: **active** (Le pare-feu est en marche).

Default: **deny (incoming)** (Tout ce qui entre est bloqué par défaut).

13250/tcp ALLOW IN Anywhere (L'exception pour notre administration est bien là).

```
Status: active
Logging: on (low) #Journal des logs pour voir ce qui a été bloqué (mode low)
Default: deny (incoming) #tout refuser en entrée, allow (outgoing) #tout autoriser en sortie pour les MAJ
par exemple, disabled (routed) #routage désactivé, Debian agira comme un hôte final
New profiles: skip

13250/tcp      ALLOW IN    Anywhere
80/tcp        ALLOW IN    Anywhere
53/tcp        ALLOW IN    Anywhere
53/udp        ALLOW IN    Anywhere #Obligatoire pour répondre aux requêtes DNS
13250/tcp (v6)  ALLOW IN    Anywhere (v6)
80/tcp (v6)    ALLOW IN    Anywhere (v6)
53/tcp (v6)    ALLOW IN    Anywhere (v6)
53/udp (v6)    ALLOW IN    Anywhere (v6)
```


Installation de Pi-Hole

Et voilà de ce côté tout est configuré, on a un serveur avec un accès à distance et sécurisé. On peut passer à la suite du projet, installer pi-hole ! (car après tout c'était le but de la manœuvre).

Voici la commande pour lancer le script d'installation :

```
curl -sSL https://install.pi-hole.net | bash
```

A partir de ce moment vous voyez apparaître une framboise sur votre appareil et quelque soit la saison, elle fera plaisir cette framboise. C'est l'aboutissement de vos efforts, ensuite c'est un processus d'installation logique (similaire à celui de Debian au tout début).

Etape 1 :

Le script commence par mettre à jour les dépôts (`apt update`) et vérifier si les dépendances nécessaires sont installées (ex: `php`, `lighttpd`, `curl`).

C'est le moment où Pi-hole vérifie si le système est sain.

Etape 2 :

Le script va vous demander sur quelle interface vous souhaitez installer Pi-hole (ce sera probablement `ens33` ou `eth0`) pour rappel la connexion sur la VM est en bridged, du coup la VM fait croire à Debian qu'elle est sur un réseau câblé.

Il va vous confirmer l'IP statique que l'on a configuré. Vérifiez bien avant de valider.

Etape 3 :

Pi-hole n'est pas un fournisseur internet, c'est un filtre. Il doit donc demander aux serveurs DNS mondiaux où se trouvent les sites si le domaine n'est pas bloqué.

Google, Cloudflare ou Quad9. (Sélectionnez selon la préférence, Cloudflare 1.1.1.1 est souvent privilégié pour la vie privée, Quad9 9.9.9.9 a également une très bonne réputation à ce niveau).

Etape 4 :

Ensuite Pi-Hole propose des listes par défaut (notamment `StevenBlack`).

C'est ici que s'opère la magie : le Pi-hole télécharge les listes de domaines publicitaires et malveillants connus, il vous sera également possible d'en rajouter par la suite.

Par défaut cette première liste doit comporter 83.000 noms de domaines sur blacklist. Quand j'ai commencé le projet c'était 78.000 de mémoire, c'est le moment idéal pour rappeler qu'il est crucial pour le projet de mettre ces listes à jour et je vous explique comment un peu plus bas..

Etape 5 :

A l'étape suivante, on va vous demander si vous souhaitez installer l'interface d'administration (le dashboard Web).

Bien évidemment pour contrôler votre pi-hole, ajouter de nouvelles listes, voir les statistiques, les appareils connectés et l'efficacité du projet on dit oui. Et pendant 5 minutes on a le droit de s'estimer administrateur réseau surpuissant à chaque fois qu'on voit apparaître les statistiques.

Etape 6 :

Pi-Hole demande s'il doit reporter les requêtes DNS dans un journal de logs

Je les ai activé mais je n'ai pas encore utilisé cette fonction pour le moment je plaide coupable, c'est utile surtout pour comprendre pourquoi un site ne s'affiche pas et c'est la continuité de cet apprentissage.

Etape 7 :

À la fin, le script affiche une fenêtre grise, sur un écran bleu, très importante. **Prenez une capture d'écran ou notez précieusement les informations suivantes :**

1. **L'URL d'accès :** (<http://192.168.1.126/admin>).
2. **Le mot de passe administrateur :** Il est généré automatiquement. C'est le sésame pour accéder au dashboard. **Si vous ne le notez pas, vous serez bloqués.**

Et voici enfin l'interface web. C'est ici que l'on constate l'efficacité du serveur pi-hole, que l'on voit quels appareils sont connectés et que l'on ajoute des listes types blacklist, whitelist, des exceptions...



Petit Tips

Changer le mot de passe web

Si vous voulez un mot de passe simple à retenir sans le rendre simple à forcer :

```
sudo pihole setpassword
```

```
nicolas@Pi-Hole:~$ sudo pihole setpassword
[sudo] password for nicolas:
Enter New Password (Blank for no password):
```

En tant qu'admin sys 101 on se doit de rappeler ceci : on évite les 123456, c'est dommage d'être rigoureux sur la sécurité de la porte pour y mettre une serrure de boîte aux lettres.

Et si jamais vous voulez faire machine arrière au moment de changer de mot de passe, faites juste un **Ctrl + C** pour arrêter l'action en cours.

Vérifier l'état du service (pour confirmer que tout tourne)

```
pihole status
```

```
nicolas@Pi-Hole:~$ pihole status
/opt/pihole/utls.sh: line 100: local: FTL_PID_FILE: readonly variable
[✓] FTL is listening on port 53
  [✓] UDP (IPv4)
  [✓] TCP (IPv4)
  [✓] UDP (IPv6)
  [✓] TCP (IPv6)

[✓] Pi-hole blocking is enabled
```

Même si le dashboard vous permet de faire énormément, le terminal reste très utile pour toutes les commandes qu'on a vu ci-dessus.

Puis une fois par semaine, un petit `sudo pihole -g` mettra à jour la totalité de vos listes. Je ferai une MAJ de la doc avec le processus d'automatisation de cette commande grâce à **cron**.

Ajouter des listes :

Pour finir, vous avez, comme je le disais plus haut, la possibilité de rajouter des listes. En ce qui me concerne j'en ai pour le moment rajouter trois de plus.

Pour cela, allez dans **Lists**, ajoutez l'url de liste dans **Addresses** et cliquez sur Add Blocklist pour les blacklist et Add Allowlist pour les whitelist

https://raw.githubusercontent.com/hagezi/dns-blocklists/main/wildcard/light.txt	Une liste orientée FR
https://v.firebog.net/hosts/AdguardDNS.txt	Une liste orientée Bannière
https://v.firebog.net/hosts/Easyprivacy.txt	Pour les données de télémétrie (oui il y a débat là)

Pi-hole

Status

- Active
- 4.0 q/min
- Load: 0.00 / 0.00 / 0.00
- Memory usage: 11.2 %

MAIN

- Dashboard
- Query Log

GROUP MANAGEMENT

- Groups 1
- Clients 0
- Domains 0 0
- Lists 250 575 4

DNS CONTROL

- Disable Blocking <

SYSTEM

- Settings <
- Tools <

DONATE

- Donate

<<

hostname: Pi-Hole

≡

Subscribed lists group management

Add a new subscribed list

Address:

URL

Comment:

List description (optional)

Group assignment:

Default

Hints:

- Please run `pihole -g` or update your gravity list [online](#) after modifying your lists.
- Multiple lists can be added by separating each *unique* URL with a space or comma
- Click on the icon in the first column to get additional information about your lists. The icons correspond to the health of the list.

Add blacklistAdd allowlist

Subscribed lists

Show 10 entriesSearch:

Previous

1

Next

			Address		Status	Comment	Group assignment	
☐			https://raw.githubusercontent.com/StevenBlack/hosts/master/hosts		Enabled	Migrated from /etc/piho	Default	
☐			https://raw.githubusercontent.com/hagezi/dns-blocklists/main/wildcard/light.txt		Enabled	FR	Default	
☐			https://v.firebog.net/hosts/AdguardDNS.txt		Enabled	Bannières	Default	
☐			https://v.firebog.net/hosts/Easyprivacy.txt		Enabled	Télé-métrie	Default	

This is a blacklist
Click for details about this list

Previous1Next

Je ferai une autre documentation sur la maintenance du serveur lorsque j'aurais un peu plus d'expérience dans le domaine avec bien évidemment une utilisation des logs car c'est important de savoir faire, mais en informatique c'est d'autant plus important de comprendre pourquoi ça marche et ça ne marche pas. Ce projet est juste un premier pas, pour moi mais peut être pour vous aussi. N'hésitez pas à aller toujours plus loin.

Merci d'avoir lu jusqu'à là, n'hésitez pas à me faire des retours et j'espère que cette documentation permettra à des novices comme moi de travailler sur des projets concrets.

A très bientôt,
Nicolas C.